

HirePilot
Project Report

By
Hinal Diwani – AF05210103

INDEX

Sr.no	Topic	Page no
1	Title of Project	1
2	Acknowledgement	3
3	Abstract	4
4	Introduction	5
5	Objective	6
6	System Analysis	7
7	System Design	22
8	Screenshots	32
9	Coding	39
10	Testing	46
11	Report	47
12	Future Scope	48
13	Conclusion	48
14	Bibliography	49
15	References	49

Acknowledgement

The project "HirePilot" is the Project work carried out by

Name

Enrollment No

Hinal Diwani

AF05210103

Under the Guidance.

We are thankful to my project guide for guiding me to complete the Project. His suggestions and valuable information regarding the formation of the Project Report have provided me a lot of help in completing the Project and its related topics.

We are also thankful to my family member and friends who were always there to provide support and moral boost up.

Abstract

The project called HirePilot is about creating a web application to help candidates practice technical and HR interviews efficiently, safely, and in an interactive environment. Traditional interview preparation relies heavily on static question lists or expensive manual coaching sessions, which lack immediate objective evaluation, adaptive timing, and structured performance insights. This system replaces conventional preparation methods with an intelligent digital platform that automatically generates interview questions, monitors candidate responses, and provides comprehensive multi-dimensional evaluations.

This portal lets users register, log in, select target domain roles (e.g., Python Developer, Java Developer, Web Developer, Data Analyst, Data Scientist), specify experience tiers (Fresher, 1–2 Years, 3–5 Years, 5+ Years), and choose question difficulty levels (Easy, Medium, Hard). During the mock interview, candidates respond using either text or integrated browser voice recognition (Web Speech API) within a real-time countdown timer. The system leverages an evaluation engine combining Google Gemini AI API with a robust local NLP fallback to analyze response correctness, technical relevance, completeness, clarity, and communication quality.

To preserve session integrity, the portal incorporates automated proctoring using browser Page Visibility API tab-switch detection, automatically tracking violations and terminating non-compliant sessions. The platform is responsive, intuitive, and works seamlessly across desktops and mobile devices. It offers dynamic analytics dashboards, historical session logs, detailed diagnostic breakdowns, and personalized topic recommendations, empowering job seekers to systematically refine their interview readiness.

1. Introduction

In today's age of Information Communication and Technology, we are surrounded by technology at every moment. From the moment we wake up to the time we go to bed, technology plays a crucial role in our lives. One of the most important aspects of this technological revolution is the development of web applications. These applications enable us to access information from anywhere, anytime, and at a low cost. Information has become one of the most valuable resources in the modern world, and web applications, such as AI-powered career tools and online assessment portals, provide candidates with instant, personalized guidance.

Our project aims to develop an HirePilot website that makes realistic mock interview practice and automated performance evaluation accessible to job seekers at any time. The portal allows candidates to stay prepared for competitive technical and HR interviews in a timely and dynamic manner. Additionally, artificial intelligence and natural language processing functionalities are integrated into the portal, enabling automated scoring, identification of candidate strengths and weak areas, and instant feedback generation based on their answers. This adds depth to candidate preparation and enhances analytical insights for continuous career development.

1.1 Objective of the Present Work

The objectives of this project are as follows:

- o **To develop a web application for an HirePilot website that can prepare candidates for technical and HR job interviews.**
- o To provide role-specific and experience-tailored interview question sets in real-time.
- o To deliver automated response evaluation with multi-dimensional scoring (correctness, relevance, technical depth, clarity, completeness).
- o To integrate real-time voice input and dynamic timing per question type.
- o To enforce strict session integrity and security using tab-switch detection.
- o To generate detailed post-interview diagnostic feedback, strengths, weak areas, and actionable recommendations.
- o To create a dynamic, user-friendly platform that allows adding new question banks and evaluating answers seamlessly.

System analysis

3.1 PROBLEM DEFINITION

Many job seekers do not have an easy, accessible way to practice realistic technical and HR interviews or receive objective feedback on their answers. Standard preparation materials consist of static question-and-answer lists, which fail to evaluate candidate communication, correctness, or completeness under timed conditions. Additionally, traditional human mock interviews require scheduling, professional mentors, and significant monetary cost.

This project solves these problems by allowing candidates to customize practice sessions by role, experience tier, and difficulty, while using artificial intelligence to evaluate answers instantly. The system makes interview preparation faster, safer, and structured, giving job seekers a platform to practice, review score breakdowns, and improve skills efficiently.

3.2 Preliminary Investigation

Purpose

The HirePilot is designed to make interview preparation accessible anytime and anywhere through a digital platform. It aims to replace static interview guides by allowing candidates to attempt live mock sessions while providing real-time voice input, countdown timers, tab-switch monitoring, and AI-driven evaluation reports.

Benefits

The portal provides several advantages:

- Instant Mock Session Access: Candidates can start tailored mock interviews covering roles like Python, Web, Full Stack, Data Science, and HR.
- Voice & Text Response Support: Supports speech-to-text input via Web Speech API for natural spoken practice.
- Multi-Metric AI Evaluation: Evaluates answers on correctness, technical depth, relevance, completeness, and clarity.
- Session Integrity & Security: Detects tab-switching and automated policy violations to maintain test standards.

Proposed System

The proposed system offers a structured, reliable, and automated approach to mock interview assessment:

- Candidates can create accounts, configure role sessions, answer questions under dynamic timers, and view detailed analytical scorecards.

- o Automated evaluation engine combines Google Gemini API intelligence with a local NLP keyword evaluation fallback.
- o Historical dashboard tracks candidate performance trends across multiple interview sessions.
- o The portal is designed to be responsive, glassmorphic, and efficient, ensuring a smooth experience across devices.

3.3 Feasibility Study

The feasibility study evaluates whether the HirePilot project is practical, achievable, and beneficial. This assessment ensures the system can be successfully implemented within available resources, technology, and constraints.

Types of Feasibility Analysis

• **Technical Feasibility**

- o The portal is built using Python (FastAPI) for backend API routes and HTML, CSS (Glassmorphism), JavaScript for frontend.
- o The database is managed using MySQL 8.0, ensuring secure and scalable data storage for candidates, questions, and scores.
- o The system runs on Uvicorn ASGI web server, supporting asynchronous processing of interview sessions.
- o AI evaluation is integrated via Google Gemini API with local PyMySQL & Python NLP algorithms fallback.

• **Economic Feasibility**

- o Open-source technologies like FastAPI, MySQL, and Python minimize overall software development costs.
- o Automated AI scoring eliminates the high recurring expense of hiring human mock interview coaches.

• **Operational Feasibility**

- o Candidates can easily access the portal from any web browser on laptop or mobile.
- o Questions are organized by role and experience level, making navigation simple and intuitive.

• **Schedule Feasibility**

- o Development milestones are planned structuredly covering design, database creation, backend coding, frontend assembly, and testing.

- **Social Feasibility**

- o The project promotes self-confidence and career readiness among students and job seekers.
- o It provides unbiased, objective feedback, helping candidates identify skill gaps without social anxiety.
- o The system is accessible to all users regardless of location or economic background.

3.4 Project Planning

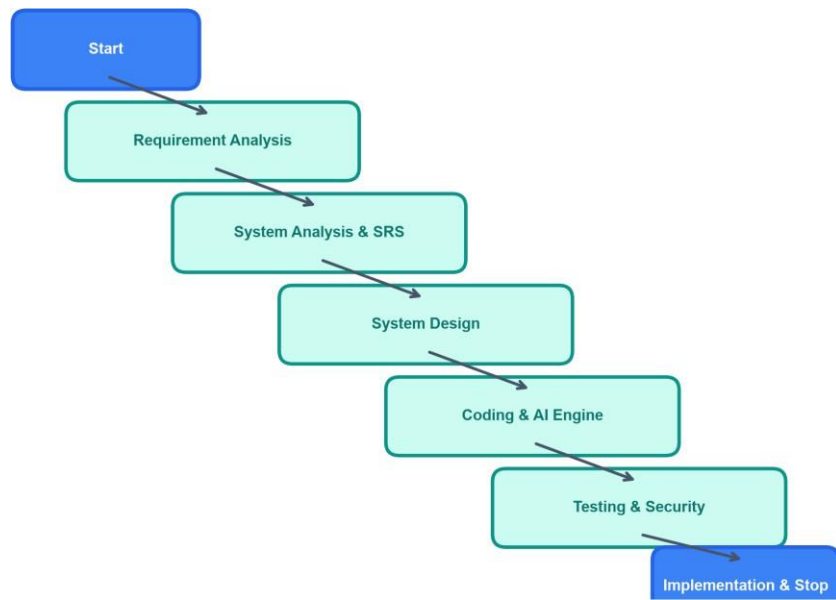
Purpose of Project Planning

Project planning ensures that the development of the HirePilot follows a structured approach. It helps define workflow, resource allocation, timelines, and security controls to deliver the system efficiently.

Phases Covered in the Plan

The planning is divided into different phases to ensure smooth execution:

1. Preliminary Investigation – Understanding project scope, domain requirements, and target users.
2. System Analysis – Identifying assessment challenges, gathering functional requirements, and defining AI fallback metrics.
3. System Design – Structuring database schema, ER diagrams, DFDs, and glassmorphic UI templates.
4. Coding – Developing FastAPI backend routes, MySQL database manager, AI evaluator, and JS interview room.
5. Security – Implementing PBKDF2 password hashing, JWT authentication, and browser Page Visibility tab-switch proctoring.
6. Testing – Performing unit testing, API route validation, response timing tests, and session security checks.
7. Implementation – Deploying the backend server, initializing MySQL seed tables, and verifying candidate flows.



SDLC Process Flow

3.5 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) outlines fundamental requirements of the AI-Powered Interview Coach System to ensure efficient functionality, security, and maintainability.

System Overview

The system allows candidates to access mock interview sessions across multiple domains. Structured into three core modules:

1. Candidate / User Module – Enables registration, login, profile management, domain role selection, and session history viewing.
2. Interview Engine & AI Evaluation Module – Manages dynamic question fetching, countdown timers, speech-to-text input, and AI answer scoring.
3. Security & Analytics Module – Enforces tab-switch monitoring, records security violations, calculates overall stats, and renders diagnostic charts.

Software & Hardware Requirements

Software Requirements

- o Frontend: HTML5, CSS3 (Vanilla Glassmorphism styling), JavaScript (ES6+ with Fetch API)
- o Backend: Python 3.11 with FastAPI framework & Uvicorn ASGI server
- o Database: MySQL 8.0 server with PyMySQL relational database connector
- o AI Engine: Google Gemini API (gemini-1.5-flash / gemini-pro) with local Python NLP fallback engine
- o Authentication: PBKDF2 password hashing & JWT (JSON Web Tokens)

Hardware Requirements

- o Processor: Intel Core i5 / AMD Ryzen 5 or higher
- o RAM: Minimum 8GB (16GB recommended)
- o Storage: At least 100GB SSD for database, audio logs, and media files
- o Connectivity: Broadband Internet access for Gemini AI API communication

3.6 Functional Requirements

1. Candidate Module

- o Register account with name, email, and secure password.
- o Login securely and receive JWT session token.
- o Select job role, experience level, interview type, and difficulty.

2. Interview & Evaluation Module

- o Fetch questions dynamically from MySQL seed bank or Gemini AI.
- o Record voice responses using Web Speech API or type text answers.
- o Receive automated multi-metric score breakdown and feedback after completion.

3.7 Software Engineering Paradigm

The development of the HirePilot follows a structured software engineering paradigm to ensure efficiency, reliability, and maintainability. The chosen paradigm streamlines development by defining clear phases while allowing iterative enhancements based on evaluation testing.

Development Model: Adapted Waterfall Model

The Waterfall Model is traditionally a linear approach, but for this project, an iterative feedback mechanism is incorporated between testing, evaluation tuning, and UI refinement. This ensures high answer precision and security compliance.

Key Adaptations in the Waterfall Model:

1. Structured Phase Progression – Each stage (requirements, design, coding, security, testing) follows a defined order.
2. Iterative Refinements – Feedback loops allow continuous tuning of evaluation algorithms and voice recognition handlers.
3. Defined Milestones – Database schemas and API routes reach complete validation before frontend integration.
4. Flexible Adjustments – Overlapping between backend API development and UI component styling enhances productivity.

Phases of Development

1. Requirement Analysis & System Study

- o Identifying candidate preparation needs, evaluation dimensions, and target interview roles.
- o Defining core backend API endpoints and MySQL table requirements.

2. System Design

- o Structuring MySQL relational tables (users, interviews, questions, answers, evaluations).
- o Designing Data Flow Diagrams (DFDs), ER diagrams, and glassmorphic UI wireframes.

3. Implementation (Coding)

- o Backend development using Python (FastAPI).
- o Frontend design using HTML5, CSS3, and JavaScript Fetch API.
- o AI integration using Google Gemini API & local NLP fallback engine.

4. Testing & Security Debugging

- o Unit testing API routes, database operations, and evaluation math.
- o Testing tab-switch monitoring and security violation handlers.

5. Deployment & Maintenance

- o Hosting on local Uvicorn ASGI server and verifying end-to-end candidate workflows.

3.8 Data Flow Diagram:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

It shows how data enter and leaves the system, what changes the information, and where data is stored.

The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

The following observations about DFDs are essential:

1. All names should be unique. This makes it easier to refer to elements in the DFD.
2. Remember that DFD is not a flow chart. Arrows in a flow chart represent the order of events; arrows in DFD represent flowing data. A DFD does not involve any order of events.
3. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge! A diamond-shaped box is used in flow charts to represent decision points with multiple exit paths of which only one is taken. This implies an ordering of events, which makes no sense in a DFD.
4. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Symbol	Name	Function
	Data Flow	Connects processes; arrow head indicates direction of data movement.
	Process	Performs data transformation of inputs to yield desired output data.
	Source / Sink (External Entity)	A source of system inputs or sink of system outputs.
	Data Store	Repository of stored data items (e.g. MySQL tables).

Symbols for Data Flow Diagrams

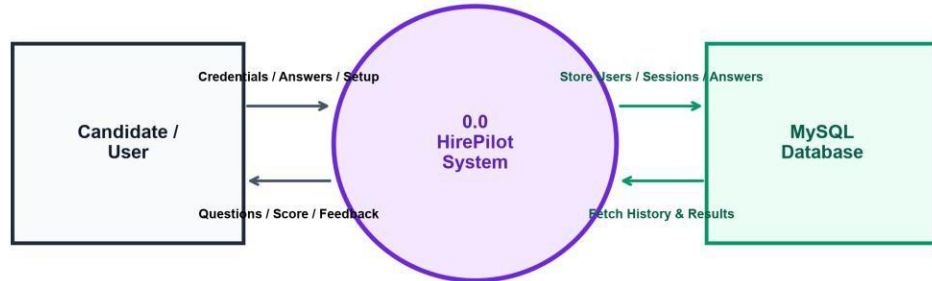
Circle: A circle (bubble) shows a process that transforms data inputs into data outputs.

Data Flow: A curved line shows the flow of data into or out of a process or data store.

Data Store: A set of parallel lines shows a place for the collection of data items. A data store indicates that data is stored which can be used at a later stage.

Source or Sink: Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

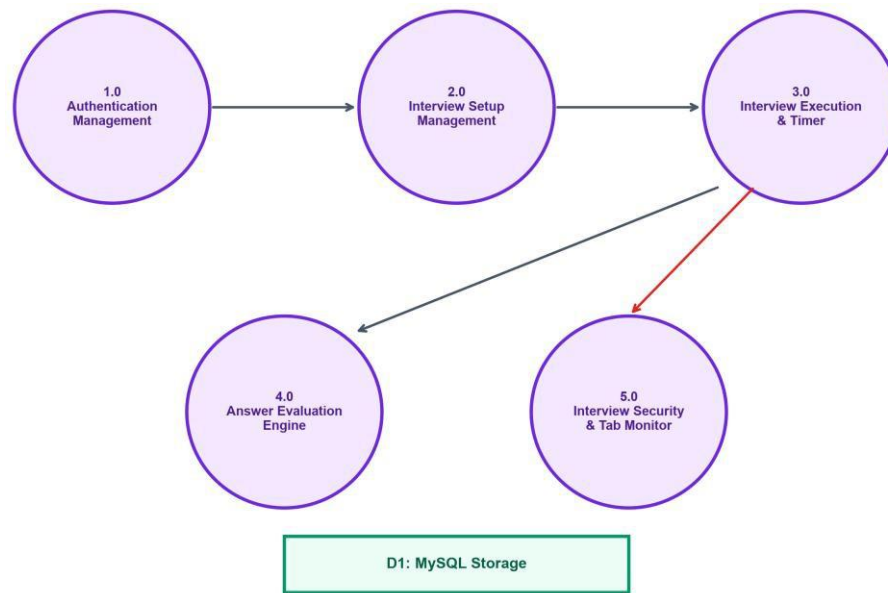
Zero Level DFD



Zero Level DFD of HirePilot

The Zero Level DFD of the HirePilot shows the main processes interacting with the central system, such as Candidate Authentication, Session Setup, Question Generator, AI Evaluation Engine, Security Proctoring, and Analytics Dashboard. It highlights the flow of data between external candidates and core modules.

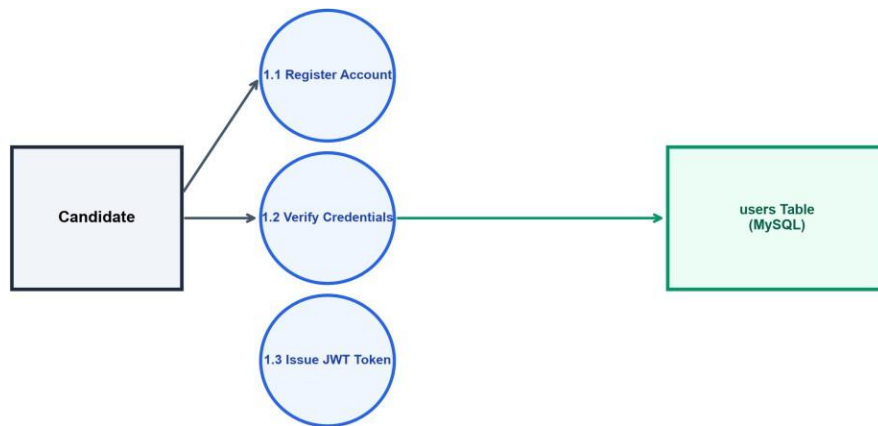
First Level DFD



First Level DFD of HirePilot

The First-Level DFD of the HirePilot shows how the system handles key functions like user registration, login authentication, interview configuration, dynamic question generation, voice/text answer processing, AI score computation, and session history management.

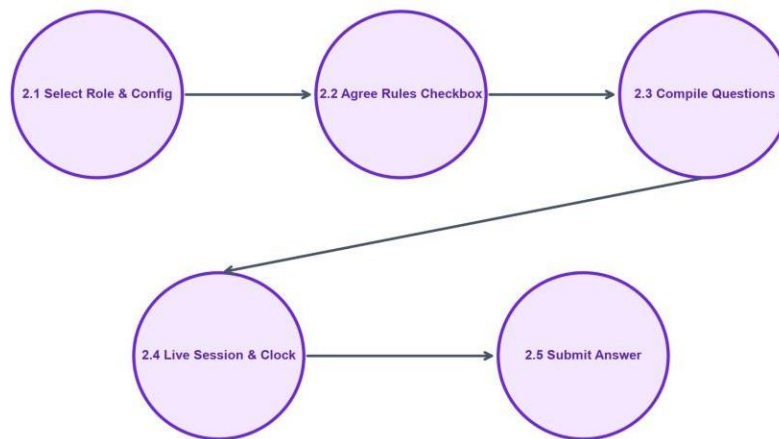
Second Level DFD - Authentication & Profile



Second Level DFD - Candidate Auth Workflow

The Second-Level DFD for Authentication shows the internal workflow of user registration, credential verification, PBKDF2 password hashing, JWT token issue, and profile updates within the MySQL database.

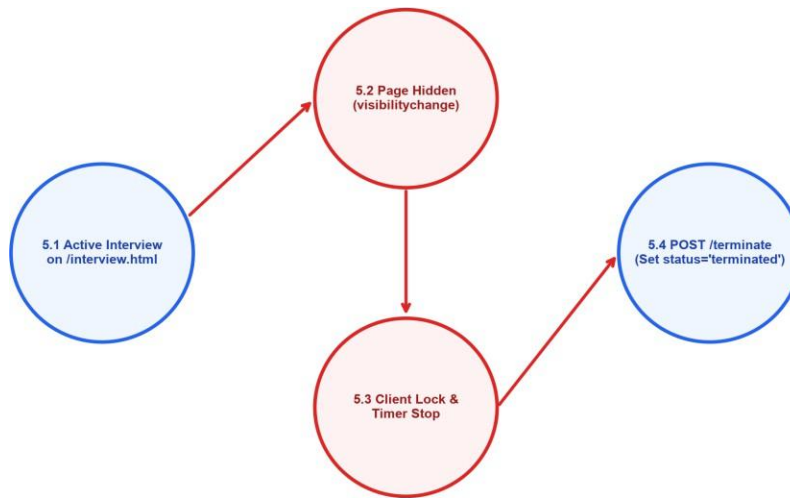
Second Level DFD - Interview Session



Second Level DFD - Interview & Question Generator Workflow

This Second-Level DFD represents the internal workflow for setting up mock interviews, querying the MySQL seed question bank or Gemini AI, initializing question timers, and receiving candidate text/voice inputs.

Second Level DFD - Security & Proctoring

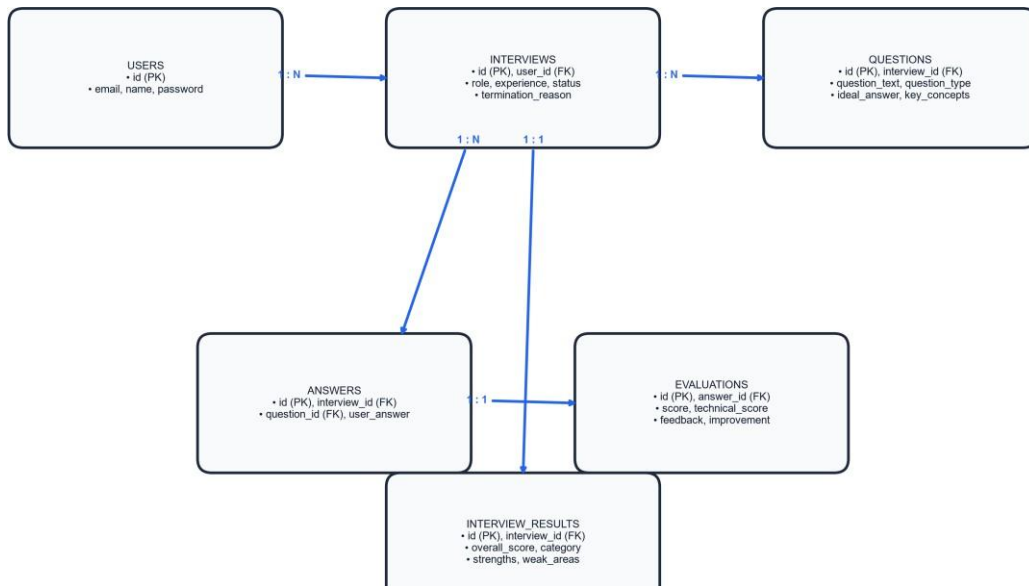


Second Level DFD - Security & Proctoring Workflow

This Second-Level DFD illustrates the security monitoring sub-system. It captures browser Page Visibility API tab-switch events, updates warning counts, logs security violations, and triggers automated session termination if rules are breached.

ER diagram

The ER Diagram of the HirePilot shows key relationships between Users, Interviews, Questions, Answers, Evaluations, Interview Results, and Question Bank tables, ensuring structured relational management.



Entity-Relationship (ER) Diagram

4. System design

4.1. Modules

1. Candidate Module:

- o Candidate Account Management – Register, login, update profile, and manage JWT tokens.
- o Interview Setup – Configure target job role, experience tier, interview type, and difficulty.

2. Interview & Evaluation Module:

- o Question Generator Engine – Fetch seed questions or request AI-generated questions.
- o Speech-to-Text Voice Engine – Capture candidate oral responses using Web Speech API.
- o Multi-Metric Evaluator – Score correctness, relevance, technical score, clarity, and completeness using Gemini AI / NLP fallback.

3. Security & Analytics Module:

- o Tab-Switch Security Proctor – Detect browser visibility changes and terminate suspicious sessions.
- o Candidate Analytics Dashboard – Display score trends, average correctness, strengths, and session history records.

4.2 DATA STRUCTURE OF ALL MODULES:

We have organized one database HirePilot for system design. It contains 7 MySQL relational tables created for user data, interviews, questions, answers, evaluations, results, and question banks:

Customized Tables Details

1. Users Table: Table name(users)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
email	varchar(150)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	

2. Interviews Table: Table name(interviews)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
role	varchar(100)	NO		NULL	
experience	varchar(50)	NO		NULL	
interview_type	varchar(50)	NO		NULL	
difficulty	varchar(50)	NO		NULL	
total_questions	int	NO		5	
overall_score	float	YES		0.0	
status	varchar(20)	YES	MUL	in_progress	
started_at	timestamp	YES		CURRENT_TIMESTAMP	
completed_at	timestamp	YES		NULL	

3. Questions Table: Table name(questions)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
interview_id	int	NO	MUL	NULL	
question_text	text	NO		NULL	
question_type	varchar(50)	YES		Technic.	
difficulty	varchar(50)	YES		Medium	
question_order	int	NO		1	
ideal_answer	text	YES		NULL	
key_concepts	text	YES		NULL	

4. Answers Table: Table name(answers)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
interview_id	int	NO	MUL	NULL	
question_id	int	NO	MUL	NULL	
user_answer	text	YES		NULL	
submitted_at	timestamp	YES		CURRENT_TIMESTAMP	

5. Evaluations Table: Table name(evaluations)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
answer_id	int	NO	MUL	NULL	
score	float	YES		0.0	
correctness_score	float	YES		0.0	
relevance_score	float	YES		0.0	
technical_score	float	YES		0.0	
clarity_score	float	YES		0.0	
completeness_score	float	YES		0.0	
feedback	text	YES		NULL	
ideal_answer	text	YES		NULL	
improvement_suggestion	text	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	

6. Interview Results Table: Table name(interview results)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
interview_id	int	NO	MUL	NULL	
technical_score	float	YES		0.0	
correctness_score	float	YES		0.0	
communication_score	float	YES		0.0	
completeness_score	float	YES		0.0	
overall_score	float	YES		0.0	
performance_category	varchar(50)	YES		Pending	
strengths	text	YES		NULL	
weak_areas	text	YES		NULL	
recommendations	text	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	

7. Question Bank Table: Table name(question bank)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
role	varchar(100)	NO	MUL	NULL	
interview_type	varchar(50)	NO	MUL	Technical	
difficulty	varchar(50)	NO	MUL	NULL	
question_text	text	NO		NULL	
topic	varchar(100)	YES		General	
ideal_answer	text	NO		NULL	
key_concepts	text	NO		NULL	

Database Indexing & Constraints

The database structure uses InnoDB engine with foreign keys ON DELETE CASCADE to maintain referential integrity across users, interviews, questions, answers, and evaluations. Indexes on email, user_id, interview_id, answer_id, role, and difficulty optimize query execution times under heavy candidate usage.

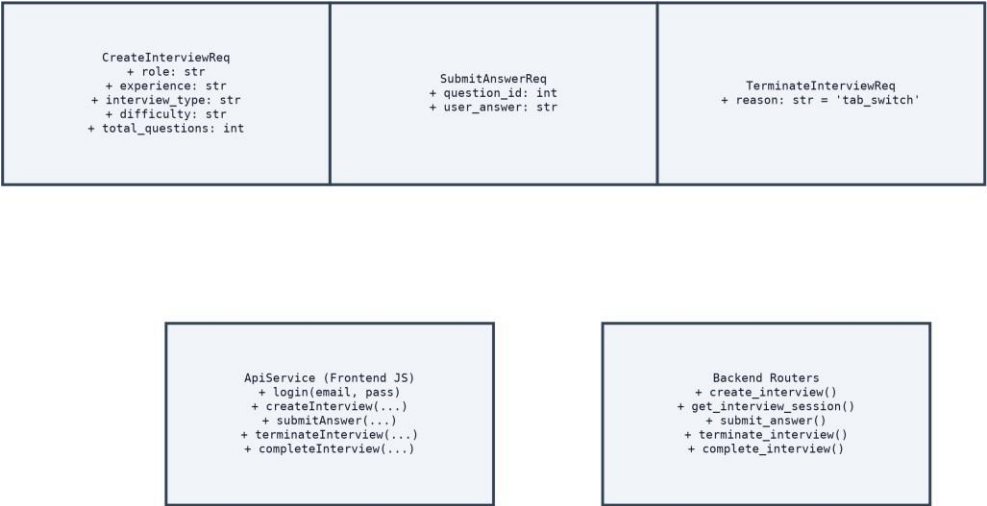
Database Schema Summary

The MySQL database HirePilot serves as the centralized persistent storage for all system state. It holds configuration tables for candidate credentials, active sessions, dynamic question lists, candidate answers, multi-metric evaluations, final session diagnostic summaries, and offline seed question banks.

Key Entity Relationships:

- One User (Candidate) can attempt Multiple Interviews (1 : M).
- One Interview contains Multiple Questions (1 : M).
- One Question receives One Candidate Answer (1 : 1).
- One Answer generates One Multi-Metric Evaluation (1 : 1).
- One Interview computes One Final Diagnostic Summary in Interview Results (1 : 1).

Relationship between tables (Class Diagram)



Class Diagram / Relational Table Schema

4.3 PROCEDURAL DESIGN:

Process logic (flowchart) of each module

4.3.1 Candidate Panel Design

In candidate panel design we have structured the main workflow for job seekers. From the index page, candidates can log in, select domain roles, configure difficulty settings, launch mock interviews, submit voice/text answers, and view instant score analytics.

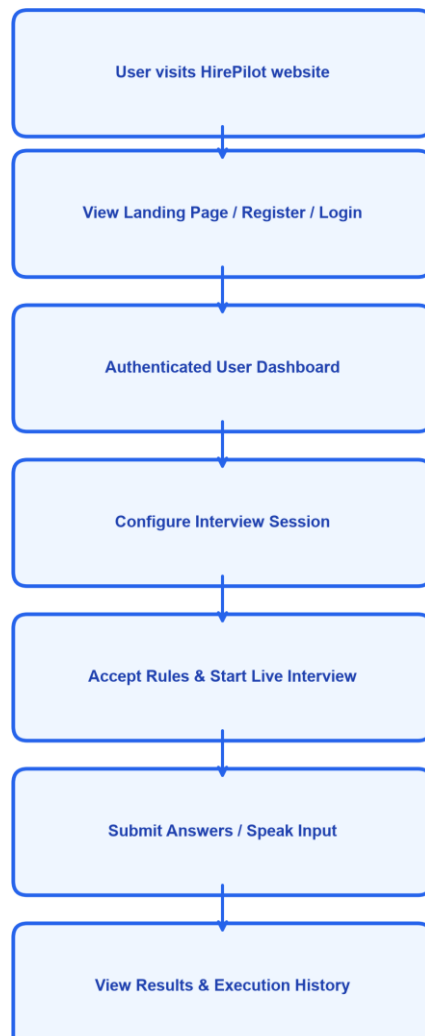
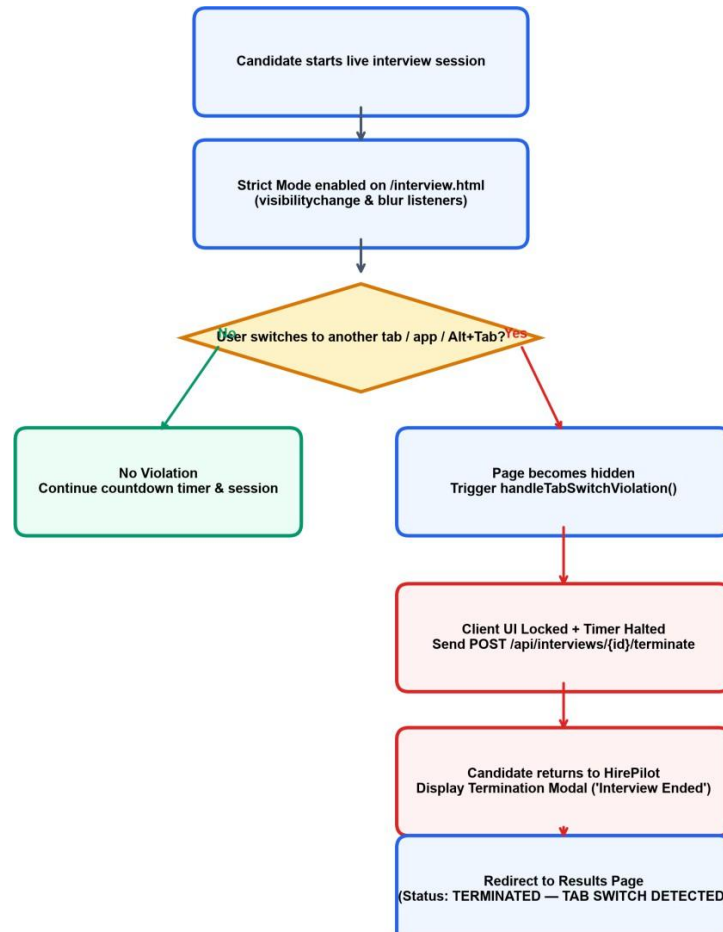


Fig. 4.1: Candidate panel flowchart part.

4.3.2 Security & Evaluation Design

We have designed automated security and AI evaluation logic to manage interview integrity. When a candidate enters the live room, browser visibility listeners track tab-switching events. If rules are breached, an alert modal warns the candidate; repeated violations immediately terminate the session and flag performance as Terminated.



Security & Evaluation Flowchart part 1.

Security Flowchart (Continued)

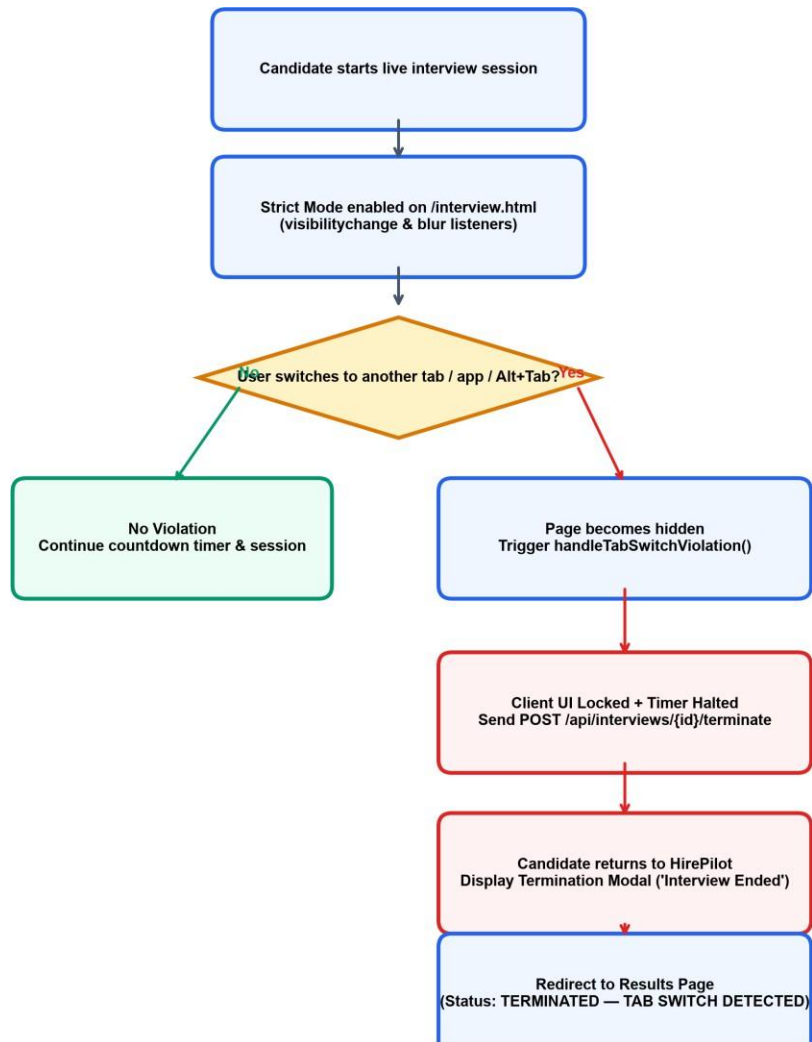
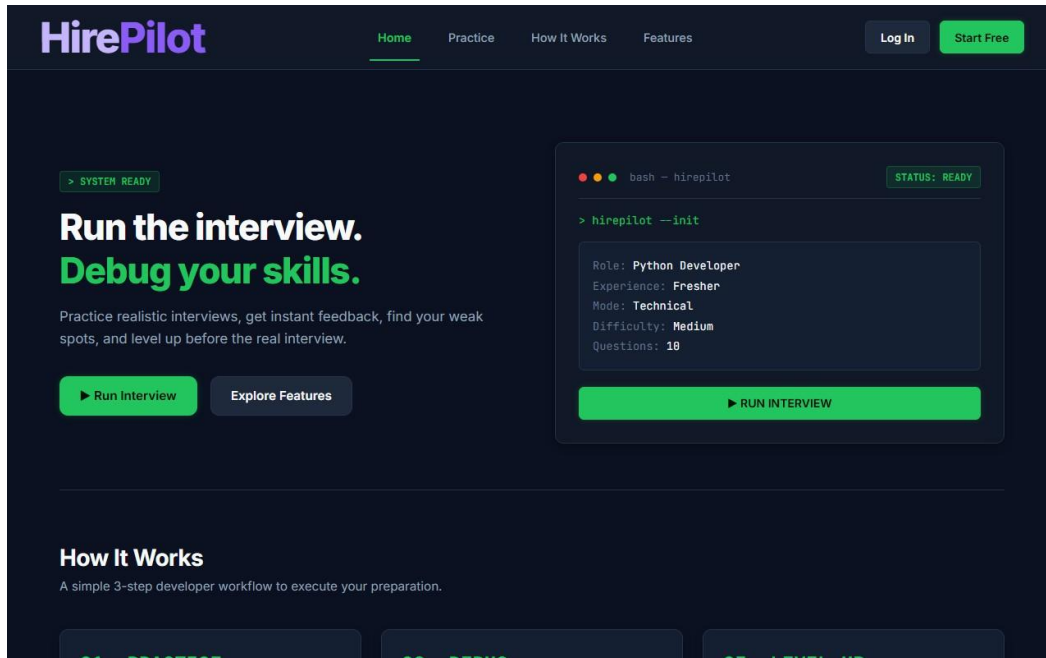


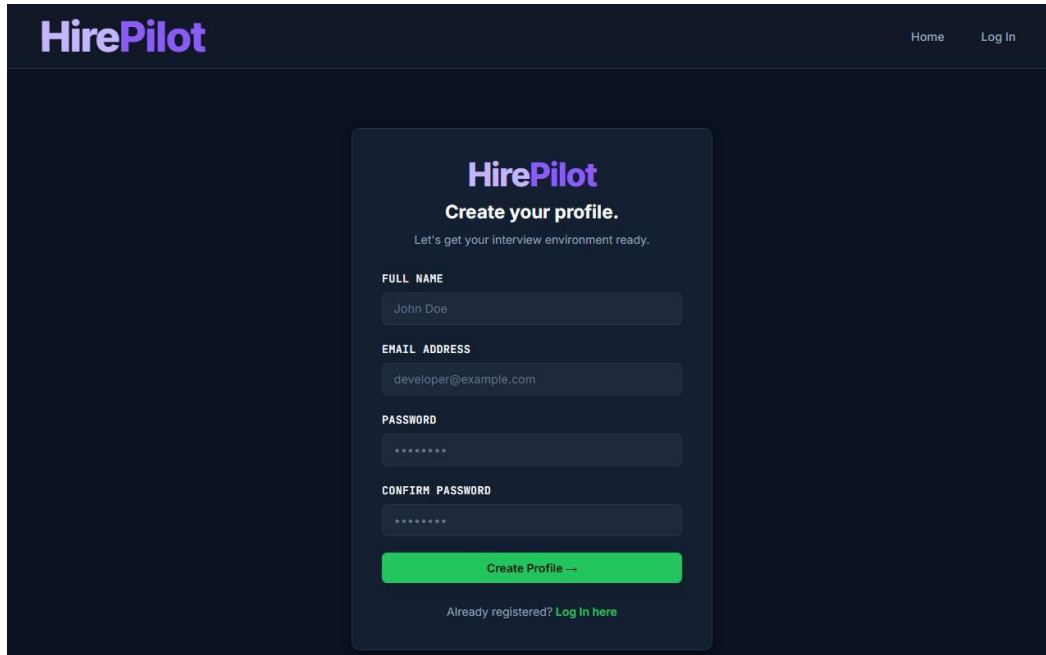
Fig. 4.2: Security & Proctoring Flow Chart.

SCREENSHOTS

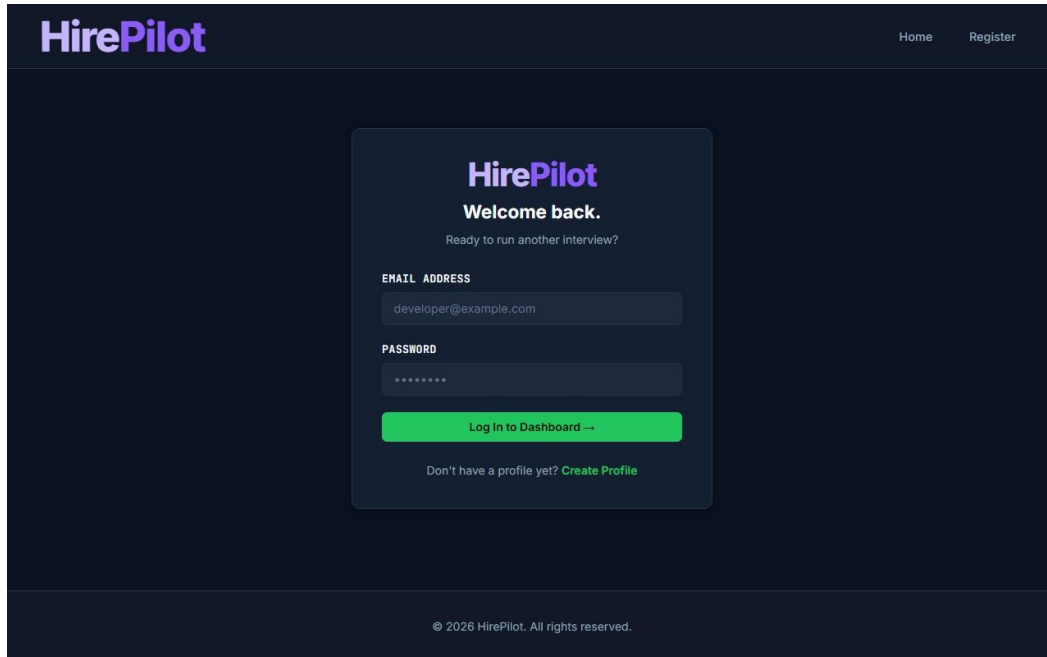
Home Page / Landing Page



Candidate Registration Page

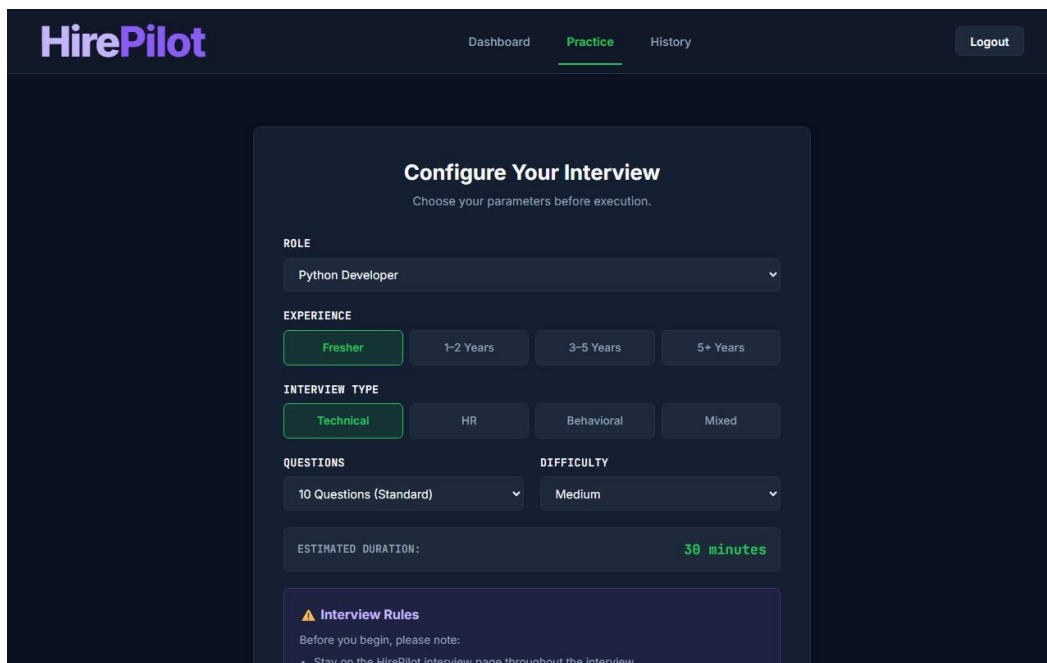


Candidate Login Page



The screenshot shows the HirePilot login interface. At the top, the 'HirePilot' logo is on the left, and 'Home' and 'Register' links are on the right. The main content area features a dark blue card with the 'HirePilot' logo, a 'Welcome back.' message, and a link 'Ready to run another interview?'. Below this are input fields for 'EMAIL ADDRESS' (containing 'developer@example.com') and 'PASSWORD' (masked with dots). A green 'Log In to Dashboard →' button is positioned below the password field. At the bottom of the card, a link says 'Don't have a profile yet? [Create Profile](#)'. The footer contains the copyright notice '© 2026 HirePilot. All rights reserved.'

Interview Setup Page



The screenshot displays the 'Configure Your Interview' page in the HirePilot dashboard. The top navigation bar includes 'Dashboard', 'Practice' (highlighted), and 'History', with a 'Logout' button on the far right. The main content area is a dark blue card titled 'Configure Your Interview' with the subtitle 'Choose your parameters before execution.' The configuration options are as follows:

- ROLE:** A dropdown menu currently showing 'Python Developer'.
- EXPERIENCE:** Four buttons: 'Fresher' (highlighted in green), '1-2 Years', '3-5 Years', and '5+ Years'.
- INTERVIEW TYPE:** Four buttons: 'Technical' (highlighted in green), 'HR', 'Behavioral', and 'Mixed'.
- QUESTIONS:** A dropdown menu showing '10 Questions (Standard)'.
- DIFFICULTY:** A dropdown menu showing 'Medium'.

Below these settings, the 'ESTIMATED DURATION:' is displayed as '30 minutes' in green text. At the bottom, a section titled '⚠ Interview Rules' contains the text 'Before you begin, please note:' followed by a bulleted list: '• Stay on the HirePilot interview page throughout the interview.'

Rules & Security Warning Modal

The screenshot shows the 'Practice' modal in the HirePilot application. At the top, there's a navigation bar with 'Dashboard', 'Practice' (highlighted), and 'History'. A 'Logout' button is in the top right. The main content area has two dropdowns: 'QUESTIONS' set to '10 Questions (Standard)' and 'DIFFICULTY' set to 'Medium'. Below these, it shows 'ESTIMATED DURATION: 30 minutes'. A section titled 'Interview Rules' with a warning icon contains the text: 'Before you begin, please note:' followed by a bulleted list: 'Stay on the HirePilot interview page throughout the interview.', 'Do not switch browser tabs or applications.', 'Do not use ChatGPT, Google, or other external resources during the interview.', and 'Switching tabs or leaving the interview page will automatically end your interview.' Below the list is a checkbox labeled 'I understand and agree to the interview rules.' which is currently unchecked. A red error message states 'Please accept the interview rules before starting.' At the bottom is a large green button labeled 'Run Interview'. The footer shows '© 2026 HirePilot. All rights reserved.'

Live Mock Interview Room (Timer & Speech Input)

The screenshot shows the 'Configure Your Interview' modal in the HirePilot application. The navigation bar is identical to the previous screenshot. The modal title is 'Configure Your Interview' with the subtitle 'Choose your parameters before execution.' The configuration options include: 'ROLE' set to 'Python Developer'; 'EXPERIENCE' with buttons for 'Fresher' (highlighted), '1-2 Years', '3-5 Years', and '5+ Years'; 'INTERVIEW TYPE' with buttons for 'Technical' (highlighted), 'HR', 'Behavioral', and 'Mixed'; 'QUESTIONS' set to '10 Questions (Standard)'; 'DIFFICULTY' set to 'Medium'; and 'ESTIMATED DURATION: 30 minutes'. At the bottom, the 'Interview Rules' section is partially visible, showing the same warning icon and the first bullet point: 'Stay on the HirePilot interview page throughout the interview.'

Security Violation / Terminated Session Page

The screenshot shows the HirePilot login interface. At the top left is the 'HirePilot' logo. At the top right are links for 'Home' and 'Register'. The main content area features a dark blue card with the following elements:

- HirePilot** logo and the text 'Welcome back.' followed by 'Ready to run another interview?'.
- An 'EMAIL ADDRESS' label above a text input field containing 'developer@example.com'.
- A 'PASSWORD' label above a password input field showing seven asterisks.
- A green message box stating 'Please log in to run an interview.'
- A prominent green button labeled 'Log In to Dashboard →'.
- A link at the bottom that says 'Don't have a profile yet? [Create Profile](#)'.

The footer of the page contains the copyright notice: '© 2026 HirePilot. All rights reserved.'

Detailed Evaluation & Feedback Breakdown Page

Candidate Analytics Dashboard

HirePilot

[Home](#) [Register](#)

HirePilot
Welcome back.
Ready to run another interview?

EMAIL ADDRESS

developer@example.com

PASSWORD

Please log in to access your dashboard.

Log In to Dashboard →

Don't have a profile yet? [Create Profile](#)

© 2026 HirePilot. All rights reserved.

Session History Records Page

The screenshot shows the HirePilot login interface. At the top, the 'HirePilot' logo is on the left, and 'Home' and 'Register' links are on the right. The main content area features a dark blue card with the following elements: the 'HirePilot' logo, the text 'Welcome back.', a subtext 'Ready to run another interview?', an 'EMAIL ADDRESS' label above a text input field containing 'developer@example.com', a 'PASSWORD' label above a password input field with six dots, a green button with the text 'Please log in to view your execution history.', a red button with the text 'Log In to Dashboard -->', and a link 'Don't have a profile yet? Create Profile' at the bottom. A footer at the very bottom reads '© 2026 HirePilot. All rights reserved.'

Candidate Profile Page

This screenshot is identical to the one above, showing the HirePilot login interface. The only difference is the text on the green button, which now reads 'Please log in to view your developer profile.' instead of 'execution history'. All other elements, including the logo, links, input fields, and footer, remain the same.

Coding

main.py

```
# backend/main.py - FastAPI Application Server
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from backend.database import get_db_connection
from backend.auth import router as auth_router
from backend.interview import router as interview_router
from backend.dashboard import router as dashboard_router

app = FastAPI(title="HirePilot API", version="2.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

app.include_router(auth_router, prefix='/api/auth', tags=['Auth'])
app.include_router(interview_router, prefix='/api/interview', tags=['Interview'])
app.include_router(dashboard_router, prefix='/api/dashboard', tags=['Dashboard'])

app.mount('/', StaticFiles(directory='frontend', html=True), name='frontend')
```

auth.py

```
# backend/auth.py - Password Hashing & JWT Manager
import jwt
import hashlib, os
from datetime import datetime, timedelta
from fastapi import APIRouter, HTTPException, Depends

SECRET_KEY = "hirepilot_secret_key_jwt"

router = APIRouter()

def hash_password(password: str) -> str:
    salt = os.urandom(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    return salt.hex() + ':' + pwd_hash.hex()

def create_access_token(data: dict, expires_delta: timedelta = timedelta(hours=8)) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + expires_delta
    to_encode.update({'exp': expire})
    SECRET_KEY = "hirepilot_secret_key_jwt"
```


question_generator.py

```
# backend/question_generator.py - AI & Seed Question Generator
from backend.database import get_db_connection
import google.generativeai as genai
import os, json

def generate_questions(role: str, experience: str, difficulty: str, count: int = 5):
    conn = get_db_connection()
    cursor = conn.cursor()
    query = '''
        SELECT id, question_text, topic, ideal_answer, key_concepts
        FROM question_bank
        WHERE role = %s AND difficulty = %s ORDER BY RAND() LIMIT %s
    '''
    cursor.execute(query, (role, difficulty, count))
    questions = cursor.fetchall()
    conn.close()
    if questions and len(questions) >= count:
        return questions
    return fetch_from_gemini_api(role, experience, difficulty, count)
```

evaluator.py

```
# backend/evaluator.py - Multi-Metric AI Evaluator
def evaluate_candidate_answer(question_text: str, ideal_answer: str, user_answer: str):
    if not user_answer or len(user_answer.strip()) < 5:
        return {
            'score': 0.0, 'correctness': 0.0, 'relevance': 0.0, 'technical': 0.0,
            'clarity': 0.0, 'completeness': 0.0, 'feedback': 'No substantive response
provided.'
        }
    # NLP Keyword Matching & Gemini Scoring Logic
    matched_keywords = [w for w in ideal_answer.lower().split() if w in
user_answer.lower()]
    kw_ratio = min(1.0, len(matched_keywords) / max(1, len(ideal_answer.split())))
    correctness = round(kw_ratio * 10.0, 1)
    return {
        'score': correctness,
        'correctness': correctness,
        'relevance': round(min(10.0, correctness + 1.5), 1),
        'technical': correctness,
        'clarity': 8.5,
        'completeness': correctness,
        'feedback': 'Good attempt. Ensure core technical terminology is explicitly
highlighted.'
    }
```

interview-setup.js

```
// frontend/js/interview-setup.js - Interview Configuration Script
document.getElementById('start-interview-btn').addEventListener('click', async () => {
  const role = document.getElementById('role-select').value;
  const experience = document.getElementById('experience-select').value;
  const difficulty = document.getElementById('difficulty-select').value;
  const token = localStorage.getItem('token');

  const response = await fetch('/api/interview/start', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${token}`
  },
    body: JSON.stringify({ role, experience_level: experience, difficulty })
  });
  const data = await response.json();
  if (data.interview_id) {
    window.location.href = `/interview.html?id=${data.interview_id}`;
  }
});
```

interview.html

```
<!-- frontend/interview.html - Live Mock Interview Room UI -->
<!DOCTYPE html>
<html lang='en'>
<head>

<title>HirePilot || Live Room</title>
<link href='../css/interview.css' rel='stylesheet' type='text/css'>
</head>
<body>
  <main class='interview-container'>
    <header class='room-header'>
      <h2 id='question-counter'>Question 1 of 5</h2>
      <div id='timer-display' class='timer'>02:00</div>
    </header>
    <section class='question-box'>
      <p id='question-text'>Loading question...</p>
    </section>
    <textarea id='answer-input' placeholder='Speak using mic or type your answer here...'></textarea>
    <div class='controls'>
      <button id='mic-btn' class='btn-mic'>🎤 Start Voice Input</button>
      <button id='submit-btn' class='btn-submit'>Submit Answer</button>
    </div>
  </main>
</body>
</html>
```

schema.sql

```
CREATE DATABASE IF NOT EXISTS hirepilot_db;

USE hirepilot_db;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE interviews (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  role VARCHAR(100) NOT NULL,
  experience VARCHAR(50) NOT NULL,
  interview_type VARCHAR(50) NOT NULL,
  difficulty VARCHAR(50) NOT NULL,
  total_questions INT NOT NULL DEFAULT 5,
  overall_score FLOAT DEFAULT 0.0,
  status VARCHAR(20) DEFAULT 'in_progress',
  started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Testing

Unit Testing: Unit testing where individual program units or object functions are tested. Here by using unit testing we focused on testing password hashing methods, JWT generation, evaluation math routines, and database PyMySQL pool functions.

Module Testing: Where this is the combination of unit programs called a module. Here we tested API route modules (auth router, interview router, dashboard router) to ensure JSON request parameters and database updates function seamlessly.

Sub-system Testing: Then we combined multiple modules (Question Generator + Voice Input + Gemini AI Evaluator) for Preliminary System Testing in our Project.

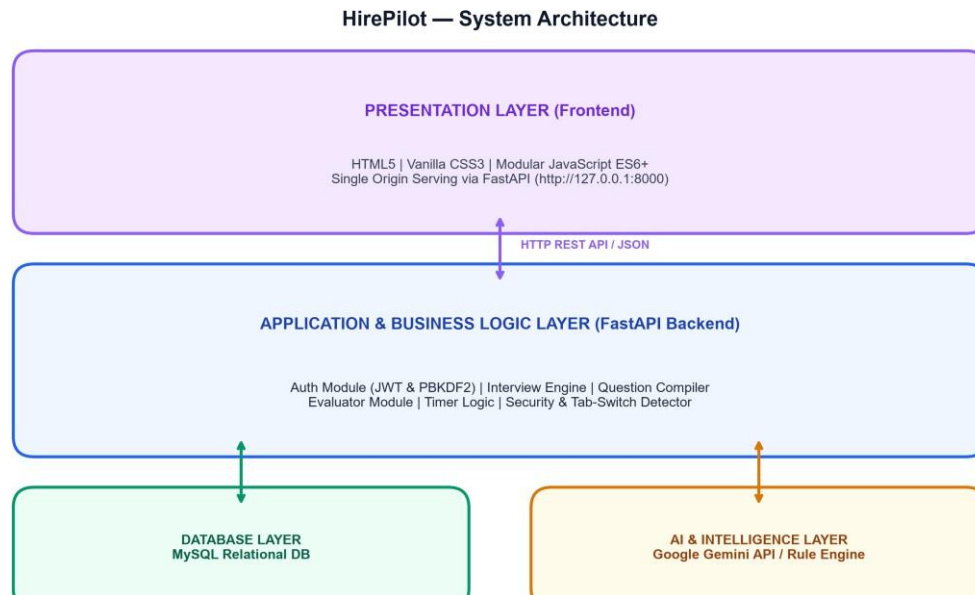
System Testing: Where it is the combination of two or more sub-systems and then tested together. Here we tested the entire system end-to-end as per requirement specifications.

Acceptance Testing: Normally this type of testing is done to verify if the system meets candidate-specified requirements. After deploying the project, candidate mock interview flows were tested to determine whether to accept the application delivery.

Report

Candidate Performance & Diagnostic Report

The report section summarizes candidate practice statistics across all completed interviews. The system architecture coordinates asynchronous FastAPI request handling, MySQL query execution, Gemini AI evaluation processing, and client-side chart rendering.



HirePilot System Architecture & Report Flow

6. Future scope

The HirePilot can be enhanced with the following features:

- o Personalized AI Recommendation Engine using candidate weakness history to automatically generate customized study plans.
- o Mobile App (iOS & Android) for flexible practice on mobile devices.
- o Real-time AI Video & Emotion Analysis via webcam to assess candidate body language and eye contact.
- o Multilingual Support to allow candidates to practice interviews in non-English languages.
- o Interactive Code Sandbox allowing live code execution during technical coding interview questions.
- o Recruiter & Enterprise Dashboard enabling corporate HR teams to assign custom interview assessments and review candidate video recordings.

Conclusion

The HirePilot offers a comprehensive and efficient solution for managing and conducting mock job interviews in a digital format. By automating the evaluation process, it enhances candidate productivity, ensures immediate feedback, and provides a user-friendly interface for both technical and HR interview preparation. The system incorporates role-based customization, ensuring secure and appropriate functionality for users. With features like speech recognition, multi-metric AI scoring, tab-switch proctoring, and diagnostic reporting, it fosters better candidate engagement and performance tracking. Designed with scalability and flexibility in mind, the system is well-equipped to adapt to future enhancements, making it a reliable and robust platform for career readiness.

BIBLIOGRAPHY

- o Web Development with FastAPI and Python by Sebastian Ramírez
- o Building Data-Driven Web Applications with MySQL by Mark Lutz
- o Designing Machine Learning Systems by Chip Huyen

REFERENCES

- FastAPI Official Documentation – <https://fastapi.tiangolo.com/>
- Python Official Documentation – <https://www.python.org/doc/>
- Google Gemini API Documentation – <https://ai.google.dev/docs>
- MySQL 8.0 Reference Manual – <https://dev.mysql.com/doc/refman/8.0/en/>
- W3C Web Speech API Specification – <https://w3c.github.io/speech-api/>
- W3C Page Visibility API Specification – <https://www.w3.org/TR/page-visibility/>